Optional

Video Reference: Java Optional in-depth

What is the problem for which Optional was introduced?

Methods generally return "null" which indicate value is not present and many times client forgot to add "null" check, which led to *NullPointerException.*

```
public class UserRepository {

public User findUserById(int id) {
            // If user not found, return null
            return null;
        }

}


public class Demo {

 public static void main(String[] args) {
        User user = new UserRepository().findUserById(1);
        // forgot to add null check
        System.out.println(user.getName()); // NullPointerException
    }

}
```

Concept && Coding

In another terms we can say:

Earlier methods does not have a way to tell the clients that response value may or may not exist.
and that forced client to add repetitive null checks *across the project* to avoid *NullPointerException*.

```
public class Demo {

 public static void main(String[] args) {
        User user = new User Repository().findUserById(1);
        // forced client to add null check to avoid any chance of NPE
         if(user != null) {
               System.out.println(user.getName());
        }
    }
}

```

Optional<T>:

- Introduced in Java8, to solve the above discussed i.e. null-return problem.
- API or methods, now can express their intention for ex:
- *Optional<User>* as return type, tells client that → return value may or may not exist.

```
 public Optional<User> findUserById(int id) {
                    .
```

```
                    .
                    .
}

public class Demo {

 public static void main(String[] args) {
        Optional<User> userOptional = new UserRepository().findUserById(1);
        if(userOptional.isPresent()) {
                User user = userOptional.get();   // extract the value
                 System.out.println(user.getName());
        }
    }
}
```

Many times, engineer ask questions:
Hey, earlier we used to put *If condition for null check*, now after using *optional, still we are using if condition* what's the advantage?

Advantages:

- Clients now do not need to guess, whether return value can be null or not. If they see Optional<T> means developer is saying, value may or may not exist.

  User findUserById(int id) // return type does NOT tell caller that null may come

  Optional<User> findUserById(int id) // API clearly says: value may be absent

Concept && Coding

- Complier force us to handle the Optional return type.

  Optional<User> u = repo.findUserById(1);

  System.out.println(u.getName()); *//this does not compile*

Null return type do not give us anything, while Optional provide many utility methods to handle the response:
- isPresent()
- ifPresent()
- orElse()
- orElseGet()

etc.....

So, at compile time itself we can avoid mistakes.

Lets see major Optional methods:

Creation of Optional object:

- Optional.of(value)

- Optional.ofNullable(value)
- Optional.empty()

Presence checking of the value from Optional object:

- isPresent()
- isEmpty() - *Introduced in Java 11*

Retrieving values from Optional object:

- get()
- orElse(default)
- orElseGet(Supplier)
- orElseThrow() - *Introduced in Java 10*
- orElseThrow(Supplier)

Transforming Values

- map(Function)
- flatMap(Function)
- filter(Predicate)

Concept && Coding

Action based methods

- ifPresent(Consumer)
- ifPresentOrElse(Consumer, Runnable) - *Introduced in Java 9*

Alternative selection

- Optional.or(Supplier<Optional>) - *Introduced in Java 9*

Stream integration

- Optional.stream() - *Introduced in Java 9*

Lets see one by one:

Optional<T> class structure:

```
public final class Optional<T> {

  //Holds the actual value and its type is Generic.
    private final T value;

```

```
  /*
  Static Optional object of Unknown type (? Wild card) which holds null value.
  Whenever we need to return Empty Optional object, it does not create
  new object every time, it just use this.
  */
    private static final optional<?> EMPTY = new Optional<>(null);

  //Constructor is private, so we directly can not call:new Optional(value)
    private Optional(T value) {
        this.value = value;
    }
              .
              .
              .
}
```

Creation of Optional object:

1. **Optional.of(value):**

Optional class (Framework)

```
public static <T> Optional<T> of(T value) {

    if (value == null) {
        throw new NullPointerException("Value must not be null");
    }
    return new Optional<>(value);
}

```

Concept && Coding

- It's a static method, used to created non empty Optional object.
- If null is passed, it will throw NullPointerException.

Usage example:

```
public Optional<String> getUserName(String value) {
            return Optional.of(value);
}
```

2. **Optional.ofNullable(value):**

Optional class (Framework)

```
public static <T> Optional<T> ofNullable(T value) {
        return value == null ? EMPTY : new Optional<>(value);
}

```

- It's a static method, used to created Optional object which *may* or *may not* contain a non-null value.

Usage example:

```
public Optional<String> getUserName(String value) {
            return Optional.ofNullable(value);
}
```

3. **Optional.empty():**

returns a shared, immutable empty Optional instance representing “no value present”.

Optional class (Framework)

```
private static final Optional<?> EMPTY = new Optional<>(null);

public static<T> Optional<T> empty() {
            Optional<T> t = (Optional<T>) EMPTY;
            return t;
}
```

Usage example:

```
public void getUserName() {
        //both point to same empty object
        Optional<String> s = Optional.empty();
        Optional<Integer> i = Optional.empty();
}
```

Presence checking of the value from Optional object:

1. **isPresent():**

Concept && Coding

Optional class (Framework)

```
public final class Optional<T> {

    private final T value;

    private Optional(T value) {
        this.value = value;
    }

     public boolean isPresent() {
           return value != null;
     }
               .
               .
               .
}
```

It checks whether the Optional object holds a non-null value, and returns true if value exist else return false.

Usage example:

```
//MyStringUtil class
public Optional<String> getUserName() {
           return Optional.of("val");
}

//Client usage
MyStringUtil myObj = new MyStringUtil();
Optional<String> output = myObj.getUserName();
if(output.isPresent()) {
         //do something
```

```
}
```

2. **isEmpty():**

Optional class (Framework)

```
public boolean isEmpty() {
        return value == null;
}

```

- Introduced in Java 11
- It is simply the logical negation of isPresent() and provides a more readable way to check if an Optional object is empty.

Usage example:

```
MyStringUtil myObj = new MyStringUtil();
Optional<String> output = myObj.getUserName();
if(output.isEmpty()) {
         //do something
}
```

- Before Java11, we need to do *!output.isPresent()*

Concept && Coding

Retrieving values from Optional object:

1. **get():**

Optional class (Framework)

```
public T get() {
         if (value == null) {
                throw new NoSuchElementException("No value present");
         }
         return value;
 }

```

- If value is present returns it, else throws exception.

Usage example:

```
//usecase:1
Optional<String> userNameOptional= Optional.of("Shrayansh");
if(userNameOptional.isPresent()) {
         String output = userNameOptional.get();
}

//usecase:2
Optional<String> userNameOptional= Optional.empty();
  String output = userNameOptional.get();  //NoSuchElementException
```

```

```

2. **orElse(default):**

Optional class (Framework)

```
public T orElse(T other) {
            return value != null ? value : other;
}
```

- If value is present returns it else return the provided default.
- So no need for putting additional if condition: "outputOptional.isPresent()", we can pass default value, so when value is null, default value is returned.

Usage example:

```
//usecase:1
Optional<String> userNameOptional= Optional.of("Shrayansh");
 String output = userNameOptional.orElse("defaultUserName"); //returns Shrayansh

//usecase:2
Optional<String> userNameOptional= Optional.empty();
 String output = userNameOptional.orElse("defaultUserName"); //returns defaultUserName
```

Concept && Coding

3. **orElseGet(Supplier):**

Optional class (Framework)

```
public T orElseGet(Supplier<? extends T> supplier) {
              return value != null ? value : supplier.get();
}


@FunctionalInterface
public interface Supplier<T> {
        T get();
}

```

- When default value required custom computation, then we need to use this "orElseGet"
- If value is present returns it else execute the lambda expression (Supplier) to compute the default value.

Usage example:

```
//usecase:1
Optional<String> userNameOptional= Optional.of("Shrayansh");
String output = userNameOptional.orElseGet(() -> {
                                  return "default value";
                          }); //returns Shrayansh


//usecase:2
```

```
Optional<String> userNameOptional= Optional.empty();
//lambda expression will execute and returns default value
String output = userNameOptional.orElseGet(() -> {
                                return "default value";
                        });

```

4. **orElseThrow():**

- Introduced in Java 10
- If value is present return it else throw *NoSuchElementException*

Optional class (Framework)

```
public T orElseThrow() {
    if (value == null) {
            throw new NoSuchElementException("No value present");
     }
     return value;
}
```

Usage example:

```
//usecase:1
Optional<String> userNameOptional= Optional.of("Shrayansh");
 String output = userNameOptional.orElseThrow(); //returns Shrayansh


//usecase:2
Optional<String> userNameOptional= Optional.empty();
 String output = userNameOptional.orElseThrow(); //throws NoSuchElementException


```

Concept && Coding

5. **orElseThrow(Supplier):**

- If value is present return it, else it will execute our custom logic to create and throw custom exception.

Optional class (Framework)

```
public <X extends Throwable> T orElseThrow(Supplier<? extends X> exceptionSupplier) throws X {
            if (value != null) {
                        return value;
            } else {
                   throw exceptionSupplier.get();
             }
   }

```

Usage example:

```
//usecase:1
Optional<String> userNameOptional= Optional.of("Shrayansh");
String output = userNameOptional.orElseThrow(() -> new IllegalArgumentException("user not found")); //returns Shrayansh


```

```
//usecase:2
Optional<String> userNameOptional= Optional.empty();
String output = userNameOptional.orElseThrow(() -> new IllegalArgumentException("user not found")); //throws IllegalArgumentException


```

Transforming Values

Optional map(Function), flatMap(Function) and filter(Predicate) is conceptually similar to

Stream map(Function), flatMap(Function) and filter(Predicate).

So do check Stream first.

We have already covered Stream and its sub topics like in depth:

- **Map(Function)**
- **flatMap(Function)**
- **Filter(Predicate)**

Kindly check it out, if there is any doubt with it.

Preview unavailable

Concept && Coding

Difference is:

- Stream : transforms 0...N elements (collection)
- Optional: transforms 0 or 1 element, as Optional is a wrapper of only 1 value.

1. **map(Function):**

- It is used to transform an element.
- Like stream its an intermediate operation, in Stream, input is stream and after map operation i.e transformation output is also stream.
- Similarly in.

  Optional, input is Optional<T> and after map operation i.e transformation output is also Optional<U>

Optional class (Framework)

```java
public <U> Optional<U> map(Function<? super T, ? extends U> mapper) {
        //Mapper i.e Lambda expression,should not be null
             Objects.requireNonNull(mapper);
        //Value should not be null
             if (isEmpty()) {
                         return empty();
                } else {
                       /* Runs the functional interface method
                       (via lambda expression) and its output is wrapped
                       again in Optional.
                      */
                       return Optional.ofNullable(mapper.apply(value));
                 }
 }
```

Usage example:

```java
//MyStringUtil class
public Optional<String> getUserName() {
        return Optional.of("Shrayansh");
}

//client usage:

MyStringUtil myObj = new MyStringUtil();
Optional<String> outputOptional = myObj.getUserName();
Optional<Integer> lengthOptional = outputOptional.map((String val)-> {
                                    int len = val.length();
                                    return len;
                                 });
System.out.println(lengthOptional.get()); //prints 9

```

Concept && Coding

2. **<u>flatMap(Function):</u>**

- Problem with Optional map is:

  what if the function (Lambda expression) is also returning the Optional?
  Then we need nested Optional which is not recommended.

  For ex:

```java
MyStringUtil myObj = new MyStringUtil();
Optional<String> outputOptional = myObj.getUserName();
Optional<Optional<Integer>> lengthOptional = outputOptional.map((String val)-> {
                                                int len = val.length();
                                                 return Optional.of(len);
                                            });

```

<u>So whenever is this scenario, use flatMap, instead of map:</u>

Optional class (Framework)

```java
public <U> Optional<U> flatMap(Function<? super T, ? extends Optional<? extends U>> mapper) {
                        Objects.requireNonNull(mapper);
                        if (isEmpty()) {
                                return empty();
                         } else {
                                Optional<U> r = (Optional<U>) mapper.apply(value);
                                return Objects.requireNonNull(r);
                         }
}
```

Usage example:

```java
MyStringUtil myObj = new MyStringUtil();
Optional<String> outputOptional = myObj.getUserName();
Optional<Integer> lengthOptional = outputOptional.flatMap((String val)-> {
                                                int len = val.length();
                                                return Optional.of(len);
                                        });
System.out.println(lengthOptional.get()); //prints 9

```

3. **filter(Predicate):**

- In stream, filter purpose is to select particular values from collection based on condition matching.

```java
Stream<String> nameStream = Stream.of("HELLO", "EVERYBODY", "HOW", "ARE", "YOU", "DOING");

Stream<String> filteredStream = nameStream.filter((String name) -> name.length() <= 3);

List<String> filteredNameList = filteredStream.collect(Collectors.toList());

System.out.println(filteredNameList); // [HOW, ARE, YOU]

```

Concept && Coding

- In Optional, filter keeps the value only if it match the condition, else it will return Optional.empty().

Optional class (Framework)

```java
public Optional<T> filter(Predicate<? super T> predicate) {
                Objects.requireNonNull(predicate);
                if (isEmpty()) {
                        return this;
                } else {
                        return predicate.test(value) ? this : empty();
                }
    }

```

Usage example:

```java
//usecase-1
Optional<String> outputOptional = Optional.of("Shrayansh");

outputOptional = outputOptional.filter((String val)-> {
                            return val.length() > 10;
                    });
```

```

System.out.println(outputOptional.isPresent()); //prints false


//usecase-2
Optional<String> outputOptional = Optional.of("Shrayansh");

outputOptional = outputOptional.filter((String val)-> {
                            return val.length() <= 9;
                    });

System.out.println(outputOptional.isPresent()); //prints true

```

Action based methods:

1. **ifPresent(Consumer):**

- If value is present, execute the given action, else do nothing.

Optional class (Framework)

```
public void ifPresent(Consumer<? super T> action) {
        if (value != null) {
                action.accept(value);
        }
}
```

Usage example:

Concept && Coding

```
//usecase-1
Optional<String> userNameOptional = Optional.of("Shrayansh");
userNameOptional.ifPresent((String val)-> {
                                    //logic here
                                     System.out.println("executing my action here");
                                });
//executing my action here will get printed

//usecase-2
Optional<String> userNameOptional = Optional.empty();
userNameOptional.ifPresent((String val)-> {
                                    //logic here
                                     System.out.println("executing my action here");
                                });
//value is null, so nothing will execute here

```

2. **ifPresentOrElse(Consumer, Runnable):**

- Introduced in Java9

Optional class (Framework)

```
public void ifPresentOrElse(Consumer<? super T> action, Runnable emptyAction) {
            if (value != null) {
                action.accept(value);
            } else {
```

```
                emptyAction.run();
            }

```

Usage example:

```
//usecase-1
Optional<String> userNameOptional = Optional.of("Shrayansh");

userNameOptional.ifPresentOrElse(
        (String name) -> System.out.println("User found: " + name), // Consumer
        () -> System.out.println("User not found")            // Runnable
);

//Output: User found: Shrayansh

//usecase-2
Optional<String> userNameOptional = Optional.empty();

userNameOptional.ifPresentOrElse(
        (String name) -> System.out.println("User found: " + name), // Consumer
        () -> System.out.println("User not found")            // Runnable
);

//Output: User not found


```

Concept && Coding

Alternative selection

1. **Optional.or(Supplier<Optional>):**

- Introduced in Java9
- Similar to orElseGet(), If value is present returns it else execute the lambda expression (Supplier) to compute the default value.
- But one difference is:

  orElseGet : returns T (value)

  or: return Optional<T>

Optional class (Framework)

```
public Optional<T> or(Supplier<? extends Optional<? extends T>> supplier) {
            Objects.requireNonNull(supplier);
            if (isPresent()) {
                    return this;
            } else {
                Optional<T> r = (Optional<T>) supplier.get();
                return Objects.requireNonNull(r);
            }
}

```

Usage example:

```java
//orElseGet() example:
Optional<String> userNameOptional= Optional.empty();

//lambda expression will execute and returns default value
String output = userNameOptional.orElseGet(() -> {
                                return "default value";
                            });


//or() example:
Optional<String> userNameOptional= Optional.empty();

//lambda expression will execute and returns another Optional object
Optional<String> output = userNameOptional.or(() -> {
                                    return Optional.of("default value");
                                    });


//or() another usecase:
Optional<String> userNameOptional= Optional.of("Shrayansh");
//returns original Optional object itself, lambda expression will not execute
Optional<String> output = userNameOptional.or(() -> Optional.of("default value"));


```

Realistic use case of "or"

```
Optional<User> user =
            findFromCache()
            .or(() -> findFromMainDatabase())
            .or(() -> findFromBackupDB());

Execution stops as soon as a value is found.

```

Concept && Coding

Stream integration

1. **Optional.Stream():**

- Introduced in Java9
- Convert Optional into Stream.
- If value is empty, it creates an empty stream.

Optional class (Framework)

```java
public Stream<T> stream() {
        if (isEmpty()) {
            return Stream.empty();
        } else {
            return Stream.of(value);
        }

```

Usage example:

```java
Optional<String> nameOpt = Optional.of("Shrayansh");
nameOpt.stream()
      .forEach((String name) -> System.out.println(name)); // prints shrayansh
```

But one questions comes, Optional only holds 1 value, why we need to convert it to stream?

My understanding is:
When we are working with Stream and invokes a method, and if that method returns Optional<T>, then to plug Optional into Stream pipeline, this is required.

For example:

Without Optional.stream()

```

public class UserDetails {

    private String email;

    public UserDetails(String email) {
        this.email = email;
    }

    public Optional<String> getEmail() {
        return Optional.ofNullable(email);
    }
}

//client usage

List<UserDetails> users = Arrays.asList(
                                    new UserDetails("a@gmail.com"),
                                    new UserDetails(null),
                                    new UserDetails("b@gmail.com"),
                                    new UserDetails(null)
                                );

//If you notice, that now within Stream, we are dealing with Optional,
//and with more complex use-cases this also get complicates.

List<String> emails =
                        users.stream()
                        .map((UserDetails user) -> user.getEmail())
                        .filter((Optional<String> mail) -> mail.isPresent())
                        .map((Optional<String> mail) -> mail.get())
                        .collect(Collectors.toList());

System.out.println(emails); //prints [a@gmail.com, b@gmail.com]

```

Concept && Coding

So, using Optional.Stream(), we convert the Optional to Stream itself.

```
List<UserDetails> users = Arrays.asList(
                                    new UserDetails("a@gmail.com"),
                                    new UserDetails(null),
                                    new UserDetails("b@gmail.com"),
                                    new UserDetails(null)
                                );

List<String> emails =
                        users.stream()
                        .map((UserDetails user) -> user.getEmail())
                        .flatMap((Optional<String> mail) -> mail.stream())
                        .collect(Collectors.toList());

```

```
System.out.println(emails); //prints [a@gmail.com, b@gmail.com]

```

Where not to use Optional:

1. **Class variable**

```
class User {
    private Optional<String> name;  // bad design
}

```

Every access now requires:

```
user.getName().orElse("default")
```

This will break existing framework:

- JSON serialization
- Lombok
- Hibernate/JPA etc.

2. **Class variable**

```
public void createUser(Optional<String> email) // bad design
{
 ...
}
```

Concept && Coding

Now, it will create confusion:

What if I send like this:

```

createUser(Optional.of("Shrayansh)); //value is present
createUser(Optional.empty()); //value is missing
createUser(null);  //optional is null itself , now we first need to do Null check for optional, before even doing isPresent() or get(), else it will throw NPE

```

3. **Serializable class**

```
class UserDTO implements Serializable {
        private Optional<String> name;  //  bad design
}

```

- JSON serialization might not work out of the box. As most serializers(Jackson) expect standard getter, setter methods.
- But suppose even if it works, the JSON payload may also include internal structure of Optional, which increase our payload size.

Instead of : { "name": "A" }

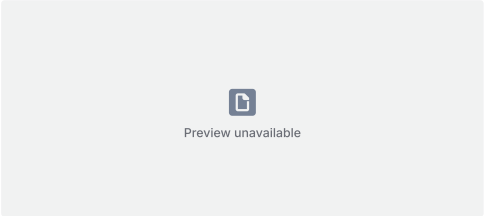
We may get: { "name": { "present": true, "value": "A" } }

4. **Serializable class**

```
public Optional<String> getNameFromDB(...)  // not recommended at DAO layer, even though we are using Optional in return type only

```

Lets understand with this:




```
ResultSet rs = statement.executeQuery("SELECT mail FROM User WHERE id = 2");

if (rs.next()) {
      String mail= rs.getString("mail");
       System.out.println(mail);  //this will print null
}

```

JDBC return "null" for SQL NULL, so contract is clear: SQL NULL → Java null

Concept && Coding

But, if we change it and say, say whenever any DB value is NULL, I want to represent it with Optional.

So, what we are doing is: SQL NULL → Java null → Optional.empty()

```
public Optional<String> getNameFromDB(int id) {
            ResultSet rs = statement.executeQuery("SELECT mail FROM User WHERE id = 2");
                String name = rs.getString("mail"); // may be null
                return Optional.ofNullable(name);
}

```

Now:

- Every DAO method forces Optional handling
- Every service must handle Optional

This spreads Optional everywhere, So instead of isolating null handling, you pollute the whole call chain (from Controller to DAO layer)

My recommendation:

Best place to use Optional is --⟶ Service Layer, because:

- It can take decision, what to pass to client ultimately. No matter what value you get from DB or by calling other external service, your service layer perform business logic and take decision, what goes out of the application.

- We can get control the usage of Optional for specific business flow only.

Concept && Coding